Week 8: Deep Learning

Activation Functions, CNNs, RNNs & Best Practices

MDST Project: Heart Failure Survival Analysis

2026

Outline

1. Recap: Perceptron & MLP
2. Activation Functions
3. Regularization
4. Convolutional Neural Networks
5. Recurrent Neural Networks
6. MLP vs CNN vs LSTM: Comparison
7. Best Practices

Recap: From Perceptron to Multi-Layer Perceptron

Perceptron (1958):

- Single neuron: $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$
- Can only learn **linearly separable** patterns
- Binary step activation: 0 or 1

Limitation: Cannot solve XOR — proved to kill neural net research in 1969!

Multi-Layer Perceptron (MLP):

- Stack multiple layers of neurons
- **Hidden layers** learn intermediate representations
- With non-linear activations: *universal function approximator*

Key insight: Depth + non-linearity = power to learn *any* function

Forward pass: $\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right)$

Activation Functions: Why They Matter

Without non-linear activations, stacking layers collapses to a single linear transformation:

$$\mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\mathbf{x})) = \mathbf{W}_{\text{eff}}\mathbf{x} \quad \longrightarrow \text{still linear!}$$

Activation function σ introduces non-linearity between layers.

Desirable properties:

- Non-linear
- Differentiable (for backprop)
- Doesn't saturate (avoids vanishing gradients)
- Computationally cheap

Common choices:

- **ReLU** — hidden layers
- **Sigmoid** — binary output
- **Softmax** — multi-class output
- **Tanh** — zero-centered

Activation Functions: Formulas & Trade-offs

Activation	Formula	Range	Pros	Cons
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Probability output	Vanishing gradients
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero-centered	Vanishing gradients
ReLU	$f(z) = \max(0, z)$	$[0, \infty)$	Fast, no saturation	Dead neurons
Leaky ReLU	$f(z) = \max(\alpha z,\ z),\ \alpha \approx 0.01$	$(-\infty, \infty)$	No dead neurons	Slight overhead
ELU	$f(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$	$(-\alpha, \infty)$	Smooth, mean ≈ 0	Slight overhead

Practical Recommendation

Hidden layers: start with ReLU **Output (binary):** sigmoid **Output (multi-class):** softmax

Vanishing Gradient Problem

What happens during backprop?

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \prod_{l=2}^{L} \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{h}^{(l-1)}}$$

Each term includes $\sigma'(z)$. For sigmoid/tanh:

- $\sigma'(z) \leq 0.25$ (sigmoid), ≤ 1 (tanh)
- Product of many small values $\to 0$
- **Early layers get no gradient**

Why ReLU helps:

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

- Gradient $= 1$ when active
- No squashing for positive values
- **Dead neuron**: if $z \leq 0$ always, gradient $= 0$ forever
- Fix: Leaky ReLU (αz instead of 0)

The Overfitting Problem

Bias-Variance Trade-off:

State	Train	Test
Underfitting	Poor	Poor
Good fit	Good	Good
Overfitting	Excellent	Poor

Regularization adds a penalty term to the loss to discourage overly complex models.

Main regularization strategies:

1. **L1 / L2** weight penalties
2. **Dropout** — random neuron masking
3. **Early Stopping** — monitor val loss
4. **Batch Normalization** — stabilize activations
5. **Data augmentation** (more data)

L1 and L2 Regularization

L2 Regularization (Weight Decay):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \sum_i w_i^2$$

- Penalizes *large* weights uniformly
- Shrinks all weights toward zero
- Weights rarely become exactly zero
- Default choice; often called `weight_decay`

L1 Regularization (Lasso):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \sum_i |w_i|$$

- Creates **sparsity**: drives weights to exactly zero
- Acts as implicit **feature selection**
- Useful when many features are irrelevant

Elastic Net: $\lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$ — combines both

Rule of thumb

λ too small $\rightarrow$ no effect. λ too large $\rightarrow$ underfitting. Tune with cross-validation.

Dropout

Idea: During each training step, randomly *zero out* neurons with probability p.

How it works:

- Each neuron independently masked: $m_i \sim \text{Bernoulli}(1-p)$
- Forward pass: $\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}$
- Gradients only flow through unmasked neurons
- At **test time**: use all neurons, scale by $(1-p)$

Typical rates:

- Fully connected: $p = 0.5$
- CNNs: $p = 0.3$
- Reduce if underfitting

Why it works:

1. Forces **redundant representations**: no single neuron can dominate
2. Equivalent to training 2^n thinned networks and averaging
3. Acts as an **ensemble method**
4. Prevents co-adaptation of neurons

In PyTorch

`nn.Dropout(p=0.5)`

Place *after* activation, *before* next layer

Batch Normalization & Early Stopping

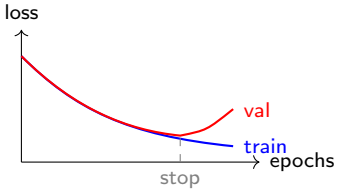
Batch Normalization:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \cdot \gamma + \beta$$

- Normalizes each layer's inputs to $\approx \mathcal{N}(0, 1)$
- γ, β are *learned* (preserves representational power)
- Allows **higher learning rates**
- Reduces sensitivity to initialization
- Acts as a regularizer (adds training noise)

Early Stopping:

- Monitor **validation loss** each epoch
- If val loss increases for N consecutive epochs: **stop**
- Restore weights from best checkpoint
- Simple yet very effective



What is a CNN?

CNNs exploit **spatial/local structure** in data using two key ideas:

1. **Local Connectivity (Receptive Fields)**
   - Each neuron connects to a *small local region*
   - Not all inputs — reduces parameters massively
   - Nearby inputs share more context
2. **Weight Sharing**
   - Same filter (kernel) applied *everywhere*
   - A filter detecting an edge works anywhere in the image
   - 3×3 filter × 64 filters = **576** params (vs millions for FC)
3. **Hierarchical Feature Learning**
   - Layer 1: edges, textures
   - Layer 2: corners, curves
   - Layer 3+: faces, objects, patterns

CNN for Heart Failure?

Tabular data reshaped as **1D sequence**:

- Each feature → one channel
- Local feature interactions (e.g., ejection fraction + creatinine)
- Use `Conv1d` layers

CNN Architecture Components

Convolutional Layer:

- Apply K filters of size k
- Each filter detects one pattern type
- Output: K feature maps
- Learned parameters: $K \times k \times C_{\text{in}}$

Pooling Layer:

- **Max pool**: largest activation wins
- **Avg pool**: smooth features
- Reduces spatial size
- Translation invariance
- No learnable parameters

Fully Connected Layer:

- Standard linear layer
- At the *end* of the CNN
- Combines all learned features
- Outputs class logits

Typical 1D CNN for tabular data:

$$\text{Input} \rightarrow \underbrace{\text{Conv1d} \rightarrow \text{BN} \rightarrow \text{ReLU}}_{\times n} \rightarrow \text{AdaptiveMaxPool} \rightarrow \text{Flatten} \rightarrow \text{Linear}$$

How Convolutions Work: Receptive Field

Receptive field: the input region that influences one output neuron.

1D Example (kernel size = 3):

`Input:`	[1]	[2]	[3]	[4]
`Filter:`	[a]	[b]	[c]	
`Output` y_1:	$a{\cdot}1 + b{\cdot}2 + c{\cdot}3$			

Stacking layers grows the receptive field:

- Layer 1 ($k = 3$): RF = 3
- Layer 2 ($k = 3$): RF = 5
- Layer 3 ($k = 3$): RF = 7

Parameter efficiency:

Detecting a 7×7 pattern:

- Fully connected: **49** parameters per output
- 3 CNN layers (k=3): $3 \times 9 =$ **27** parameters

Key formula:

$$\mathrm{RF}_l = \mathrm{RF}_{l-1} + (k - 1)$$

CNN Historical Milestones

Year	Model	Authors	Key Innovation
1998	LeNet-5	LeCun et al.	Conv + pooling + FC; first practical CNN
2012	AlexNet	Krizhevsky et al.	Deep CNN + ReLU + dropout; won ImageNet
2014	VGGNet	Simonyan & Zisserman	Depth matters: 16–19 layers of 3×3
2014	GoogLeNet	Szegedy et al.	Inception modules; multi-scale features
2015	ResNet	He et al.	**Skip connections**; enabled 152 layers
2016	DenseNet	Huang et al.	Dense connections; feature reuse
2019	EfficientNet	Tan & Le	Compound scaling for mobile/edge

Why RNNs? The Sequence Modeling Problem

MLPs have fixed-size inputs and no notion of time. But many problems are sequential:

Examples of sequential data:

- Text: "The cat sat..." (order critical)
- Time series: stock prices, ECG signals
- Speech: audio frames
- DNA: nucleotide sequences
- Patient history: medical records over time

What RNNs add:

- **Hidden state** h_t carries memory across timesteps
- **Parameter sharing** across time: same W at every step
- Handles **variable-length** sequences

For heart failure: Patient records over time form a temporal sequence; earlier measurements predict future outcomes.

Vanilla RNN: Core Equations

Forward pass at each timestep t:

$$h_t = \tanh\big(W_{ih}\, x_t + W_{hh}\, h_{t-1} + b_h\big)$$

$$y_t = W_{hy}\, h_t + b_y$$

Where:

- x_t = input at time t
- h_t = hidden state (memory)
- W_{ih}, W_{hh}, W_{hy} = **shared** weights

Vanishing Gradient Problem:
BPTT unrolls the RNN through time:

$$\frac{\partial h_t}{\partial h_{t-1}} = W_{hh}^{\top} \cdot \tanh'(\cdots)$$

For T timesteps, the gradient involves:

$$\prod_{i=1}^{T} \frac{\partial h_i}{\partial h_{i-1}} = W_{hh}^{T} \cdot \prod \tanh'$$

- $|W_{hh}| < 1$: gradients **vanish**
- $|W_{hh}| > 1$: gradients **explode**
- Long-range dependencies are lost

LSTM: Long Short-Term Memory

Key idea: Use multiplicative *gates* to control information flow, with a separate cell state C_t that avoids gradient squashing.

LSTM Gate Equations:

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Forget gate
$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Input gate
$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	Candidate
$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	Cell state
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Output gate
$h_t = o_t \odot \tanh(C_t)$	Hidden state

Why it works:

- C_t updated with **addition** (not product) $\rightarrow$ gradient highway
- $f_t \approx 1$: cell state preserved $\rightarrow$ no vanishing
- $f_t \approx 0$: old info forgotten (selective)
- Gates are *soft* (0–1), learned from data

Gradient flow via C_t:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t \approx 1$$

GRU: Gated Recurrent Unit

Simpler alternative to LSTM: combines forget + input into a single *update gate*.

GRU Equations:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad \text{Reset gate}$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad \text{Update gate}$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b) \quad \text{Candidate}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{Output}$$

GRU vs LSTM:

	LSTM	**GRU**
Gates	3	2
States	h_t, C_t	h_t
Params	More	Fewer
Performance	Similar	Similar
Best when	Large data	Small data

Rule of thumb: Start with LSTM; use GRU if data is limited or speed matters.

RNN Architectures for Different Tasks

Task type	Architecture	Input→Output	Example
Classification	Many-to-one	Sequence → label	Sentiment analysis, health risk
Seq labeling	Many-to-many	Sequence → sequence	Named entity recognition
Translation	Encoder-decoder	Sequence → sequence	Machine translation
Language model	Many-to-many	Token → next token	Text generation, GPT

Attention Mechanism (2014):
Allow decoder to attend to all encoder states:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- Q = Query (what do I need?)
- K = Key (what can I offer?)
- V = Value (actual information)

Transformer (2017):

- Self-attention *without recurrence*
- Fully parallel computation
- Solves long-range dependency
- Foundation of BERT, GPT, T5

Trend: Transformers are replacing RNNs even for sequence tasks.

Model Comparison: MLP vs CNN vs LSTM

	MLP	CNN	LSTM
Input type	Fixed-size vectors	Spatial / sequential	Temporal sequences
Key inductive bias	None	Local patterns, translation invariance	Temporal ordering, long-range memory
Parameter sharing	None	Across spatial positions	Across timesteps
Depth vs cost	Quadratic growth	Efficient via convolutions	Sequential; hard to parallelize
Heart failure use	Tabular features	Local feature interactions	Patient history over time
Complexity	Low–Medium	Medium–High	High

For this dataset

Hyperparameter Selection Strategy

Learning Rate (most critical):

- Too small: very slow convergence
- Too large: loss oscillates / diverges
- **Start**: `lr=0.001`
- Use learning rate schedulers

Batch Size:

- Small (16–32): noisy gradients, regularizes
- Large (256+): stable gradients, sharp minima
- **Sweet spot**: 32–128

Architecture:

- Start with 1–2 hidden layers
- Try 2×–4× # features in hidden layer
- Double *width* before doubling *depth*

Dropout:

- FC layers: $p = 0.5$
- CNNs: $p = 0.3$
- Reduce if underfitting
- Increase if overfitting

Training Checklist & Debugging

Before / During Training:

- ☐ Normalize inputs (StandardScaler)
- ☐ Stratified train/val/test splits
- ☐ Plot loss & accuracy curves
- ☐ Tune on *validation set only*
- ☐ Save checkpoint on best val loss
- ☐ Evaluate test set **once at the end**

Common Issues:

Problem	Likely Cause	Fix
Loss is NaN	LR too high	Decrease LR, clip gradients
Loss not decreasing	LR too low	Increase LR, check normalization
Train↓ Val↑	Overfitting	Dropout, L2, early stop
Loss oscillates	Batch too small	Bigger batch, lower LR
Val plateaus	Underfitting	More layers, longer training

Karpathy's Recipe

Start simple. Overfit one batch first (sanity check). Then regularize. Then scale.

Start Simple, Add Complexity Gradually

Recommended workflow (Karpathy & Google ML best practices):

1. **Understand the data**: distributions, outliers, missing values, class balance
2. **Baseline**: Logistic Regression — can humans solve this? Is there signal?
3. **Simple non-linear**: Random Forest or Gradient Boosting
4. **MLP (2–3 layers)**: moderate non-linearity, fewer assumptions
5. **CNN** (if local feature interactions are plausible)
6. **LSTM** (only if temporal structure exists)

Complexity ladder for heart failure dataset

`Logistic Regression` → `RF/LightGBM` → `MLP` → `CNN1D` → `LSTM`

When to stop climbing:

- When val metric stops improving
- When added complexity doesn't justify gains
- When you can't explain *why* the fancier model should help

Summary: Week 8

Key Concepts Covered:

- ✓ **Activation functions**: ReLU, sigmoid, tanh — why non-linearity matters
- ✓ **Vanishing gradients**: why sigmoid fails deep, why ReLU helps
- ✓ **Regularization**: L1/L2, dropout, batch norm, early stopping
- ✓ **CNNs**: local connectivity, weight sharing, pooling
- ✓ **RNNs/LSTMs**: hidden state, vanishing gradient solution via gating
- ✓ **Best practices**: start simple, train checklist, debugging

Formulas to Remember:

Concept	Formula
ReLU	$\max(0, z)$
L2 loss	$\mathcal{L} + \lambda \lVert \mathbf{w} \rVert_2^2$
Dropout	$\tilde{h} = m \odot h, \;\; m \sim \text{Bern}$
LSTM cell	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
Attention	$\text{softmax}(QK^\top / \sqrt{d_k})\, V$

This week's task: Implement MLP, CNN, LSTM on heart failure dataset and compare.